

Project Report: Hospital Management System

Author

- Name – Aditya Anand
- Student Roll no – 24f2003177
- Student email address – 24f2003177@ds.study.iitm.ac.in
- Personal email address – mail4adi.nnd@gmail.com
- A student passionate about developing practical solutions, with a keen interest in web development and database management.

Problem Statement

The goal of this project was to build a Hospital Management System (HMS) web application to replace traditional manual or disconnected systems. The system is designed to be used by three distinct roles: **Admins**, **Doctors**, and **Patients**. The primary objectives were to manage patient/doctor records, prevent appointment scheduling conflicts, and provide a centralized, role-based platform for all hospital interactions.

My Approach

I built the application using a **Flask** backend, an **SQLite** database, and a **Bootstrap** frontend, as specified in the project requirements.

The core of the application is a robust, role-based authentication system built with **Flask-Login**. After logging in, each user is directed to a dashboard tailored to their role.

- **Patient Role:** Patients can register for an account, log in, and edit their own profiles. Their dashboard allows them to search for doctors by specialization. They can select a doctor, view their specific availability for the next 7 days, and book an available session. Patients can also view a list of all their upcoming and past appointments, cancel upcoming ones, and view the doctor's diagnosis and prescription for completed visits.
- **Doctor Role:** Doctors can log in to their dashboard to see a list of all patients booked for their upcoming sessions. They can manage these appointments by viewing a patient's medical history, marking the appointment as "Completed" (which includes adding a diagnosis and prescription), or cancelling the appointment. This role also features a "Manage Availability" page where the doctor can set their 7-day schedule using session-based time slots (e.g., 08:00-12:00).
- **Admin Role:** The admin has full oversight of the system. The admin dashboard displays live statistics for total doctors, patients, and appointments. The admin has full CRUD (Create, Read, Update, Delete) functionality for both doctors and patients, including search functionality for both. A key security feature I implemented is the "Blacklist" system, allowing the admin to deactivate (and re-activate) any user account.
- **Core Logic:** The booking system prevents scheduling conflicts by enforcing a 40-patient limit per session. It also validates that a patient's selected time falls within a doctor's set availability.

Frameworks and Libraries Used

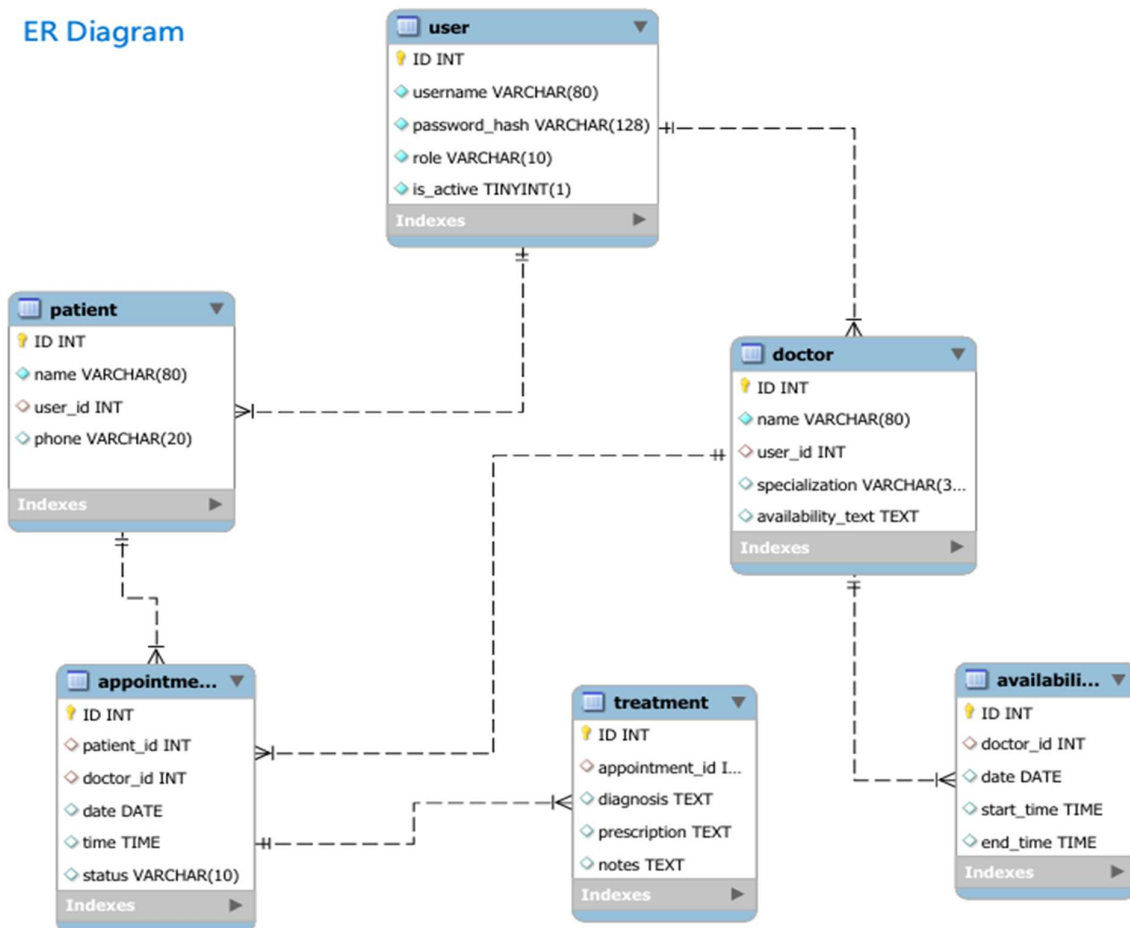
- **Backend:** Flask, Flask-SQLAlchemy, Flask-Login, Werkzeug (for password hashing)
- **Database:** SQLite
- **Frontend:** HTML, Jinja2, Bootstrap 5, custom CSS (for styling)

DB Schema Design

The database schema manages users, doctors, patients, Appointments, Treatments, and Availabilities.

- user Table: id (primary key), username(unique), password_hash, role, is_active
- patient Table: id (primary key), name, phone, user_id (foreign key to user.id)
- doctor Table: id (primary key), name, specialization, user_id (foreign key to user.id)
- availability Table: id (primary key), date, start_time, end_time, doctor_id (foreign key to doctor.id)
- appointment Table: id (primary key), date, time, status, patient_id (foreign key to patient.id), doctor_id (foreign key to doctor.id)
- treatment Table: id (primary key), diagnosis, prescription, notes, appointment_id (foreign key to appointment.id)

ER Diagram



Architecture and Features

The project utilizes an MVC-like architecture:

- `app.py`: Main controller handling routes, requests, database interaction, and template rendering.
- `models/database.py`: Defines SQLAlchemy database models.
- `templates/`: Contains dynamic HTML templates (views) using Jinja2.
- `static/`: Stores static assets like CSS and images.

Implemented Features

General & System Features

- **Role-Based Login System:** A secure login/logout system (using Flask-Login) that directs users to different dashboards based on their role (Admin, Doctor, Patient).
- **Password Hashing:** All user passwords are securely hashed using Werkzeug and are never stored in plain text.
- **Dynamic Navbar:** The navigation bar changes to show the correct links (e.g., "Login" vs. "Logout," "Dashboard") depending on the user's login status.
- **Mobile-Friendly Layout:** mobile-responsive design that ensures all navbar links are visible on all screen sizes.
- **Blacklist System:** Blacklisted users are prevented from logging into the system.

Admin Features

- A central dashboard that displays live statistics for the total number of doctors, patients, and appointments.
- Admin can add/view/edit/delete any doctor/patient's profile.
- **Search:** Search for patients by **name** or **username**.
- **Blacklist/Re-Activate:** Blacklist any doctor or patient as well as Re-activate any blacklisted user, restoring their login access from the management pages.
- View a complete list of all appointments in the system.
- View the final diagnosis/report for any completed appointment.

Doctor Features

- A dashboard that shows two separate lists: "Upcoming Appointments" and "Recently Completed Appointments."
- **Manage 7-Day Availability:** Doctors can add and delete availability slots for the next 7 days.
- **Complete an Appointment:** Doctors can select a booked appointment and move to a "Complete" page. On this page, they add a **diagnosis**, **prescription**, and **notes**, which marks the appointment as "Completed."
- **Cancel an Appointment:** Doctors can cancel an upcoming appointment from their dashboard.

- **View Patient History:** Before an appointment, a doctor can click "View History" to see a full list of that patient's past diagnoses and prescriptions.

Patient Features

- **Self-Registration:** A public-facing page for new patients to create their own accounts.
- **Patient Dashboard:** A central hub to access booking, appointment history, and profile editing.
- **Edit Profile:** Patients can update their own full name and phone number.
- **Search for Doctors:** Patients can search the list of doctors by specialization.
- **Appointment Booking:**
 - **Step 1:** Select a doctor, then select a date to "Check Availability."
 - **Step 2:** The page shows a list of available sessions for that day.
- **Booking Validation:** The system prevents a patient from booking a session that is not on the doctor's set availability/a session that is already booked.
- **View "My Appointments":** A single page showing all their appointments (Booked, Completed, Cancelled) with status badges. They can also view the doctor's report of any completed appointments.
- Patients can cancel their own "Booked" appointments.

Video Presentation Link:

https://drive.google.com/drive/folders/1kVZrVmh0Ldn0qK7LuvV__KbTmPgffO-X?usp=sharing